

DOSSIÊ DE PESQUISA

Extração de Dados de Documentos com IA Generativa:

migrando de um pipeline de duas etapas para extração multimodal em passo único

Desafio Técnico — Pesquisador(a) em IA Generativa (P&D)

Danilo Alexandre Pereira

28 de junho de 2026

Sumário

Resumo Executivo

1. Introdução

2. Metodologia

- 2.1 Fontes e processo de pesquisa
- 2.2 Abordagens consideradas e descartadas
- 2.3 Uso de ferramentas de IA
- 2.4 Critérios de comparação

3. Técnicas e Modelos Avaliados

- 3.1 Técnica A — Extração multimodal em passo único (Structured Outputs)
- 3.2 Técnica B — Parsing especializado + LLM leve para preenchimento de schema
- 3.3 Técnica C — VLM open-weight auto-hospedado

4. Resultados e Experimentos

- 4.1 Nota metodológica sobre o ambiente de execução
- 4.2 Métricas utilizadas
- 4.3 Comparação de custo
- 4.4 Comparação de acurácia (benchmarks de terceiros)
- 4.5 Comparação de arquitetura e latência
- 4.6 Resultados empíricos locais nos 3 documentos do desafio
- 4.7 Tabela-síntese de trade-offs

5. Conclusão e Recomendação

6. Análise de Viabilidade de Integração

- 6.1 Custo de infraestrutura
- 6.2 Requisitos de infraestrutura
- 6.3 Riscos, limitações e mitigação
- 6.4 Considerações de LGPD e dados pessoais

Referências

Anexo — Sobre a Prova de Conceito (POC)

Resumo Executivo

Este dossiê investiga uma alternativa ao pipeline atual da nossa API de extração de dados de documentos, hoje composto por duas chamadas sequenciais a um LLM (uma para interpretação/extração e outra para formatação no schema JSON de saída). O objetivo foi encontrar uma abordagem que reduza latência e custo sem sacrificar — e, idealmente, aumentando — a acurácia de extração em cenários críticos do produto, tais como a leitura de documentos de identificação (RG, CNH), comprovantes de residência e dados estruturados complexos.

Por convenção de resumo executivo, a recomendação é adiantada no quadro abaixo. O racional completo — metodologia de pesquisa, comparação estruturada entre três técnicas candidatas e evidências (projeções por benchmark e resultados empíricos reais) — é construído de forma independente nas Seções 2 a 4 e só é retomado como conclusão formal na Seção 5.

Recomendação em uma frase

Substituir o pipeline de duas chamadas por uma **extração multimodal em passo único**, na qual um único modelo com visão nativa recebe o documento (imagem ou PDF) e devolve diretamente o JSON no schema do cliente, usando **Structured Outputs** (decodificação restrita por schema) para garantir conformidade — eliminando por completo a segunda chamada de formatação.

Por quê: nos benchmarks e na documentação técnica levantados (detalhados nas Seções 3 e 4), essa mudança de arquitetura ataca simultaneamente os quatro problemas descritos no briefing:

- **Latência:** cai de 2 chamadas de rede/inferência sequenciais para 1 — na prática, o maior determinante de latência ponta-a-ponta em pipelines de LLM é o número de round-trips, não apenas o tamanho do modelo.
- **Custo:** a tarefa por chamada fica mais simples (não exige mais interpretar e reformatar), o que permite usar um modelo "right-sized" mais barato sem perda de acurácia mensurável em tarefas de extração pura — nossa estimativa aponta uma redução da ordem de 85-90% no custo por documento (Seção 4.3).
- **Complexidade / documentos extensos:** modelos atuais aceitam PDF nativo de até 100 páginas por requisição, sem necessidade de OCR externo ou chunking manual.
- **Gráficos e imagens:** por ser nativamente multimodal (visão, não OCR de texto), o modelo interpreta gráficos e elementos visuais — uma limitação estrutural de pipelines OCR-primeiro, que demonstramos empiricamente na Seção 4.6.

O documento está estruturado como uma investigação científica: *Metodologia* (Seção 2) descreve como a pesquisa foi conduzida e quais alternativas foram descartadas; *Técnicas e Modelos Avaliados* (Seção 3) apresenta a shortlist de 3 abordagens; *Resultados e Experimentos* (Seção 4) traz os números — tanto projeções fundamentadas em benchmarks públicos quanto resultados empíricos reais, obtidos localmente, nos 3 documentos de exemplo fornecidos; *Conclusão* (Seção 5) fecha com a recomendação; e *Viabilidade de Integração* (Seção 6) traduz tudo isso em custo de infraestrutura e riscos operacionais.

1. Introdução

Nosso produto oferece uma API de extração de dados de documentos que hoje resolve o problema em duas etapas: (1) um LLM "lê" o documento e extrai as informações solicitadas em linguagem livre e (2) uma segunda chamada ao mesmo LLM (ou a um segundo modelo) reformata essa extração no schema JSON esperado pelo cliente. Este dossiê parte de um cenário fictício, proposto no desafio técnico, no qual essa API apresenta quatro classes de problema à medida que o volume e a diversidade de documentos crescem:

- **Latência:** duas chamadas sequenciais somam os tempos de fila e inferência de cada uma, dobrando (na prática, mais que dobrando, pela sobrecarga de rede) o tempo de resposta percebido pelo usuário final.
- **Custo:** Usar modelos de ponta nas duas chamadas paga duas vezes pelo overhead de um modelo grande, mesmo quando a tarefa é objetiva (como preencher campos específicos de um formulário). Isso impede a economia que modelos mais leves e focados poderiam trazer.
- **Complexidade em Layouts Densos:** Documentos semiestruturados e formulários testam os limites de pipelines que dependem exclusivamente de OCR tradicional, exigindo parsers customizados que quebram facilmente quando o layout muda.
- **Limitações visuais e Ambiguidade:** Motores OCR clássicos falham frequentemente na desambiguação de caracteres visualmente semelhantes (como confundir o número '0' e a letra 'O' em códigos e placas), pois não possuem compreensão do contexto. Além disso, gráficos, logos, assinaturas e outros elementos não textuais carregam informações cruciais que um pipeline centrado apenas em texto (OCR clássico + LLM texto-a-texto) simplesmente não enxerga.

O objetivo desta pesquisa é identificar e validar uma abordagem alternativa que resolva — ou ao menos minimize — os problemas de latência e custo, mantendo ou aumentando a acurácia de extração. Isso é avaliado usando os três perfis de documento exigidos neste desafio: um documento estruturado de identificação (Documento 1) e um documento de layout denso (Documento 2), que representam o escopo real e prioritário do produto; e, atuando como um stress-test de limites de input e robustez da arquitetura, um documento extenso com tabelas e gráficos (Documento 3, "The Claude 3 Model Family").

Como este documento está organizado

A Seção 2 descreve a metodologia de pesquisa — fontes consultadas, abordagens descartadas e o uso de ferramentas de IA no processo. A Seção 3 apresenta a shortlist de técnicas avaliadas. A Seção 4 traz os resultados: métricas usadas, projeções de custo e latência fundamentadas em preços públicos e documentação oficial (datados de junho de 2026), acurácia reportada por um benchmark independente recente, e — de forma complementar — resultados empíricos reais, obtidos localmente, ao processar os três documentos deste desafio com um pipeline de OCR clássico, que serve de contraponto tangível às limitações discutidas. A Seção 5 fecha com a recomendação, e a Seção 6 traduz a recomendação em custo e requisitos de infraestrutura.

2. Metodologia

2.1 Fontes e processo de pesquisa

A pesquisa combinou quatro tipos de fonte, priorizadas nesta ordem quando havia conflito de informação:

- **Documentação oficial dos provedores:** (Claude API/Platform Docs, OpenAI API Docs, Google Gemini API Docs e Mistral AI Docs). Foram a fonte primária para validar capacidades nativas de *Structured Outputs* e limites rígidos de *payload* (ex: resoluções máximas e limite de tamanho de imagem por requisição).
- **Benchmarks públicos independentes:** Focados em extração de documentos, em especial o *IDP Leaderboard* (Nanonets, 16 modelos avaliados em mais de 9.000 documentos reais nos benchmarks OlmOCR Bench, OmniDocBench e IDP Core) e o próprio *OmniDocBench* (Shanghai AI Laboratory / OpenDataLab, referência acadêmica em *parsing* de documentos, CVPR 2025). A análise desses painéis priorizou o desempenho dos modelos em extração estruturada e acurácia visual.
- **Repositórios *open-source* de referência:** Analisados para compreender o estado da arte de pipelines alternativos de *document parsing* (Docling — IBM Research/LF AI & Data; MinerU — OpenDataLab).
- **Análises de mercado e *pricing* atualizadas:** Checadas em junho de 2026 para contextualizar o custo real de cada opção (preço por milhão de tokens e custo por imagem processada). Esta etapa foi crucial para validar a viabilidade financeira e o *right-sizing* da arquitetura proposta, visto que preços de API mudam com maior frequência que a literatura acadêmica.

Sempre que possível, os números de acurácia foram extraídos de um único benchmark consistente (o *IDP Leaderboard*) em vez de misturar números de fontes com metodologias diferentes. Isso evita distorções e garante uma comparação justa, já que cada benchmark pondera tarefas (OCR puro, compreensão de tabelas, desambiguação visual, fórmulas) de forma distinta.

2.2 Abordagens consideradas e descartadas rapidamente

Três famílias de abordagem foram descartadas cedo, antes de entrarem na shortlist da Seção 3:

- **Fine-tuning de um modelo próprio do zero.** Treinar um modelo multimodal proprietário para extração de documentos exige um pipeline de dados rotulados em escala, infraestrutura de treinamento dedicada e um ciclo de MLOps (versionamento, reavaliação, *retraining*) desproporcional ao ganho esperado — sobretudo porque técnicas prontas de mercado (*Structured Outputs* / *constrained decoding* sobre modelos multimodais já treinados) resolvem diretamente o sintoma-raiz do problema descrito no *briefing*, que é arquitetural (duas chamadas) e não de qualidade do modelo base.
- **OCR tradicional isolado, sem modelo de linguagem.** Motores de OCR clássicos (ex.: Tesseract) continuam relevantes como componente de extração primária, mas isoladamente não produzem JSON estruturado e não reconstroem a relação de tabelas e formulários densos. Mais grave do que isso: operam sem compreensão do contexto semântico. Isso causa falhas críticas

em desambiguações puramente visuais (como confundir o número '0' com a letra 'O' na leitura de placas, códigos ou CNHs). Para virar dados utilizáveis e corrigir esses erros de contexto, precisariam de uma segunda etapa (um LLM), recriando o mesmo gargalo de duas chamadas que este dossiê busca eliminar. Por isso, tratamos o OCR clássico como *baseline* de comparação (Seção 4.6), e não como candidato final.

- **Modelos de nicho sem visão nativa (ex.: arquiteturas puramente text-layout, tipo LayoutLM clássico).** Exigem *features* de entrada pré-computadas (*bounding boxes* de um OCR externo) e um pipeline de treinamento supervisionado específico por tipo de documento. Isso reintroduz complexidade operacional exatamente onde o objetivo é simplificar, e não generaliza bem para a diversidade de *inputs* da nossa API (que varia desde documentos de identidade curtos e faturas densas até *stress-tests* de dezenas de páginas).

2.3 Uso de ferramentas de IA

Usei o Claude (Anthropic) e Gemini como principais ferramentas de apoio à pesquisa e ao desenvolvimento ao longo de todo o processo, em três frentes:

- (1) busca e leitura de documentação técnica e benchmarks atualizados — pricing de API (Claude, OpenAI, Google Gemini, Mistral), documentação de recursos como Structured Outputs e suporte a PDF nativo, e o benchmark independente IDP Leaderboard;
- (2) ajuda e revisão do código da POC (o pipeline de extração em passo único e o baseline de OCR local) e dos gráficos comparativos deste documento; e
- (3) redação assistida deste dossiê, com síntese das fontes levantadas.

A curadoria das fontes, a definição dos critérios de comparação, a decisão de quais abordagens descartar precocemente e a recomendação final (Seção 5) refletem julgamento próprio sobre os trade-offs do problema de negócio — o papel do assistente de IA foi acelerar a coleta de informação e a produção de artefatos (código, gráficos, texto), não substituir a análise crítica sobre qual solução melhor equilibra os objetivos descritos no briefing.

***Nota sobre neutralidade de fornecedor:** O ecossistema Claude aparece com mais frequência neste dossiê do que os da OpenAI, Google ou Mistral por motivos puramente operacionais e de infraestrutura local, e não por preferência de resultado. Ele foi a ferramenta de IA integrada ao ambiente de desenvolvimento disponível para testes práticos (Seção 4.1) e, por isso, serviu como implementação de referência na POC. A recomendação final deste relatório (Seção 5) foca na **técnica** — extração multimodal em passo único com Structured Outputs — e não em um fornecedor específico. Como demonstrado na Seção 3.1, os principais provedores de mercado oferecem o mesmo recurso com garantias equivalentes de conformidade de schema, e a Seção 6.3 propõe uma camada de abstração de software justamente para evitar o vendor lock-in (dependência de um único fornecedor).*

2.4 Critérios de comparação

Cada técnica avaliada na Seção 3 foi julgada nos mesmos seis critérios, definidos antes da comparação (para evitar viés de escolher métricas que favoreçam uma resposta pré-concebida):

Critério	O que mede	Por que importa aqui
Nº de chamadas / arquitetura	Quantas chamadas de rede/inferência por documento	Determinante direto de latência

Critério	O que mede	Por que importa aqui
Custo por documento	US\$ estimado por documento processado	Objetivo explícito do desafio
Acurácia de extração	Qualidade dos campos/tabelas extraídos (fonte: 3os)	Não pode piorar em relação ao baseline
Documentos extensos	Suporte nativo a dezenas/centenas de páginas	Problema citado explicitamente no briefing
Interpretação visual	Capacidade de ler gráficos/imagens não textuais	Limitação citada explicitamente no briefing
Complexidade operacional	Infra necessária (GPU própria? orquestração?)	Viabilidade de integração (Seção 6)

3. Técnicas e Modelos Avaliados

A shortlist reúne as três abordagens mais viáveis identificadas na pesquisa. As três são compatíveis entre si — na verdade, a recomendação final (Seção 5) as trata como estágios de um roadmap, não como opções mutuamente exclusivas.

3.1 Técnica A — Extração multimodal em passo único (Structured Outputs)

Mecanismo: um único modelo multimodal (com visão nativa) recebe a imagem ou o PDF do documento em uma única chamada, junto de um JSON Schema estrito. Em vez de gerar texto livre que depois precisa ser reformatado, o próprio processo de geração de tokens é restringido para só produzir saídas que já satisfazem o schema ("constrained decoding"/"structured outputs"). O resultado sai pronto no formato do cliente na mesma chamada em que o documento foi "lido".

Esse recurso está disponível, com nomes ligeiramente diferentes, nos três principais provedores: Structured Outputs / strict tool use na Claude API (compila o schema em uma gramática que restringe a decodificação, com suporte a JSON outputs e a tool calls estritas), Structured Outputs na OpenAI API, e Controlled Generation/response_schema na Gemini API. A arquitetura é a mesma nos três casos — o que muda é o modelo específico usado por trás.

Modelos candidatos para esta técnica

- **Claude Haiku 4.5 / Sonnet 4.5** (Anthropic) — Structured Outputs em GA; PDF nativo até 100 páginas/32MB por requisição; até 600 imagens por requisição.
- **GPT-5 mini / GPT-4.1 mini** (OpenAI) — Structured Outputs com JSON Schema, visão e cache de prompt nativos.
- **Gemini 2.5 Flash / 3 Flash** (Google) — Controlled Generation, contexto de 1M tokens, forte em VQA de documentos segundo o benchmark independente citado na Seção 4.

Prós

- Elimina a segunda chamada por completo: menor latência e menor custo por construção, não por otimização incremental.

- Schema garantido estruturalmente (não depende de o modelo "lembrar" de seguir o formato) — reduz retries e erros de parsing em produção.
- Visão nativa: interpreta tabelas, gráficos e imagens no mesmo passo, sem um estágio de OCR à parte. Além disso, a compreensão contextual do VLM resolve a falha clássica de desambiguação de caracteres (ex: "0" vs "O" em placas e RGs) do OCR tradicional, pois o modelo compreende o padrão esperado para aquele campo específico.
- Migração de baixo risco: mesma categoria de infraestrutura do pipeline atual (API gerenciada por HTTP), não exige subir infraestrutura de GPU.

Contras / limitações

- Structured Outputs garante conformidade de schema, não correção semântica — validação de regras de negócio (ex.: "CPF tem 11 dígitos", "total bate com a soma dos itens") continua sendo responsabilidade da aplicação.
- Em tarefas que exigem raciocínio multi-etapa muito longo sobre o conteúdo extraído (não apenas leitura), forçar uma saída 100% estruturada cedo demais pode reduzir a qualidade do raciocínio — mitigável permitindo texto livre antes da chamada de ferramenta final (ver Seção 4.5).
- Continua dependente de um provedor de API terceirizado — relevante para a discussão de LGPD na Seção 6.4.

3.2 Técnica B — Parsing especializado + LLM leve para schema

Mecanismo: um motor especializado em document parsing (não um LLM generalista) faz a leitura de layout/tabela/OCR com alta precisão geométrica, e devolve markdown ou JSON intermediário. Quando o schema final do cliente é mais elaborado que a saída nativa da ferramenta, um LLM leve (ou o próprio Structured Outputs) faz um passo adicional, barato, de mapeamento para o schema exato.

Opções identificadas na pesquisa

- **Auto-hospedadas / open source:** Docling (IBM Research, Apache 2.0, com o modelo Granite-Docling-258M), MinerU (OpenDataLab) — ambas convertem PDF/imagem em Markdown/JSON preservando tabelas, sem custo por chamada além da infraestrutura de execução (rodam até em CPU para volumes baixos).
- **APIs comerciais especializadas:** Mistral OCR 4 (US\$ 4/1.000 páginas, US\$ 2/1.000 em batch; devolve blocos com bounding boxes e confiança) e Nanonets OCR2+ (~US\$ 10/1.000 páginas), citadas no benchmark IDP como opções que equilibram custo e acurácia frente aos modelos generalistas de ponta.

Panorama estendido: outras ferramentas de Document AI e benchmarks acadêmicos

Além de Docling, MinerU, Mistral OCR 4 e Nanonets OCR2+, a pesquisa mapeou outras famílias de modelo relevantes para este problema, que não entraram na shortlist da Seção 3 por não superarem as três técnicas escolhidas nos critérios da Seção 2.4, mas que valem registro por representarem o estado da

arte atual em Document AI: **PaddleOCR-VL** (Baidu, 0,9B parâmetros), um modelo compacto especializado em parsing de layout que compete de perto com pipelines muito maiores em benchmarks de OCR; **GOT-OCR2.0**, um OCR-2.0 de propósito geral open-weight, com bom desempenho em texto denso, fórmulas e partituras/diagramas simples; e a família **Qwen2.5-VL / Qwen3-VL** (Alibaba), VLMs open-weight com suporte nativo a grounding espacial e leitura de documentos longos, referenciados novamente na Técnica C (Seção 3.3) como o candidato mais maduro para auto-hospedagem. Do lado da avaliação, além do IDP Leaderboard e do OmniDocBench (Seção 2.1), a literatura acadêmica de referência para este tipo de tarefa inclui DocVQA e InfoVQA (perguntas e respostas sobre documentos e infográficos), OCRBench (OCR generalista, incluindo cenas naturais), ChartQA (leitura de gráficos, diretamente relevante ao problema de "limitações visuais" do briefing) e TextVQA (VQA sobre texto em imagens naturais). Não usamos números desses benchmarks para a decisão final — por consistência metodológica (Seção 2.1), preferimos uma única fonte comparável (IDP Leaderboard) aos números de acurácia citados nas Seções 4.4 e 4.7 — mas eles corroboram, de forma independente, o mesmo padrão: modelos com visão nativa superam sistematicamente pipelines OCR-primeiro em tarefas que envolvem tabelas, gráficos e layout complexo, o achado central também demonstrado empiricamente na Seção 4.6.

Prós

- Nas tarefas em que o layout geométrico é o principal desafio (tabelas muito densas, formulários), ferramentas especializadas competem de perto com modelos generalistas de ponta a uma fração do custo por página.
- A opção auto-hospedada (Docling/MinerU) roda sem depender de uma API externa por chamada — atraente para volume muito alto ou para dados sensíveis (ver 3.3 e Seção 6.4).

Contras / limitações

- Quando o schema do cliente não é trivial (campos derivados, nomenclatura específica, validações), ainda é comum precisar de uma etapa adicional de mapeamento — o ganho de "passo único" da Técnica A não se realiza por completo.
- Interpretação de gráficos/imagens não textuais é mais limitada que em um VLM generalista: ferramentas de parsing são otimizadas para texto e tabela, não para "descrever o que um gráfico mostra".
- A opção auto-hospedada troca custo variável por complexidade operacional (deploy, atualização de modelo, monitoramento) — ver Técnica C, que compartilha esse trade-off em grau maior.

3.3 Técnica C — VLM open-weight auto-hospedado

Mecanismo: um modelo multimodal de pesos abertos (ex.: família Qwen3-VL, Apache 2.0) é implantado em infraestrutura própria (GPU dedicada ou GPU cloud), servido via um framework de inferência (vLLM/SGLang) com decodificação restrita por gramática (bibliotecas como Outlines/XGrammar, já integradas ao vLLM) para obter a mesma garantia de Structured Outputs da Técnica A, porém sem chamada a um provedor terceirizado.

Prós

- Custo marginal por documento tende a zero acima de um determinado volume mensal, já que o custo principal é a GPU (fixo/amortizado), não o token.
- Dados nunca saem da infraestrutura da empresa — relevante quando os documentos contêm dados pessoais sensíveis (CPF, endereço, dados financeiros — exatamente o perfil da CNH e da fatura deste desafio) e há requisitos formais de residência de dados.
- Sem dependência de disponibilidade/rate limit de terceiros.

Contras / limitações

- Exige equipe e processo de MLOps para servir, monitorar e atualizar o modelo — o oposto do objetivo de "reduzir complexidade" do briefing, se adotado prematuramente.
- Custo de infraestrutura é fixo (paga-se a GPU ligada mesmo com baixa utilização) — só compensa financeiramente acima de um volume mínimo mensal (ver cálculo de breakeven na Seção 6.1).
- Modelos open-weight de visão ficam, em geral, um degrau abaixo dos modelos de fronteira fechados nas tarefas de raciocínio visual mais difíceis (ex.: VQA sobre gráficos complexos), embora a diferença em extração pura de texto/tabela seja pequena segundo os benchmarks consultados.

Síntese da shortlist

	Técnica A (single-pass + Structured Outputs)	Técnica B (parsing especializado)	Técnica C (VLM auto-hospedado)
Nº de chamadas	1	1-2 (depende do schema)	1
Custo/doc em baixo volume	Baixo	Baixo-médio	Alto (infra fixa)
Custo/doc em altíssimo volume	Médio (linear)	Baixo	Muito baixo (amortizado)
Interpreta gráficos/imagens	Sim (nativo)	Limitado	Sim (nativo)
Complexidade operacional	Baixa	Média	Alta
Dados saem da empresa?	Sim (API terceira)	Depende da opção	Não

Tabela 1 — Síntese comparativa qualitativa das três técnicas avaliadas. Números de custo/latência detalhados na Seção 4.

4. Resultados e Experimentos

4.1 Nota metodológica sobre o ambiente de execução

Transparência sobre o que foi de fato executado, versus projetado, é parte do rigor metodológico deste dossiê. O ambiente usado para esta pesquisa é um sandbox de execução sem GPU (1 vCPU, ~4GB de RAM), sem chave de API configurada para nenhum provedor comercial de LLM (Anthropic, OpenAI ou Google) e sem acesso a repositórios de pesos de modelo (Hugging Face bloqueado pela política de rede do ambiente). Isso significa que não foi possível, a partir deste ambiente, disparar chamadas reais às APIs comerciais que fundamentam a Técnica A nem baixar e rodar um modelo open-weight (Técnica C) — mesmo os menores modelos de visão competitivos exigem GPU para latência viável, e nenhuma está disponível aqui. A mesma restrição de rede também impediu rodar localmente as ferramentas especializadas da Técnica B (Docling e MinerU dependem de pesos hospedados no Hugging Face, bloqueado neste ambiente) — por isso a Seção 4.6 usa Tesseract como baseline, não por ser a alternativa mais forte disponível, mas por ser a única executável de ponta a ponta neste sandbox. Uma comparação empírica direta entre Tesseract, Docling, MinerU e um VLM em um ambiente com acesso a rede completo e/ou GPU é a lacuna mais importante deste dossiê e o próximo passo natural de validação, apontado como recomendação concreta na Fase 1 do roadmap (Seção 5).

O próprio enunciado do desafio antecipa esse tipo de restrição de infraestrutura e autoriza, nesses casos, fundamentar a análise comparativa em benchmarks e dados de terceiros — o caminho adotado nas Seções 4.3 a 4.5. De forma complementar (e não como substituto), a Seção 4.6 apresenta resultados empíricos reais, obtidos executando, neste mesmo sandbox, um pipeline de OCR clássico (Tesseract) sobre os três documentos de exemplo do desafio — a alternativa que de fato pôde ser testada e executada localmente, servindo como baseline tangível para o argumento sobre as limitações de pipelines centrados em texto.

4.2 Métricas utilizadas

Métrica	Definição operacional	Fonte dos números
Custo por documento (US\$)	Custo estimado de tokens de entrada+saída por documento, a preço público por milhão de tokens	Páginas oficiais de pricing (Anthropic, OpenAI, Google, Mistral), consultadas jun/2026
Nº de chamadas / round-trips	Quantas requisições sequenciais de rede o pipeline precisa por documento	Análise arquitetural (Seção 3)
Acurácia de extração (IDP Core)	Score composto (0-100) de qualidade de extração de texto/tabela/campo-chave	IDP Leaderboard, Nanonets, mar/2026 (9.000+ documentos reais, 16 modelos)
Limite de páginas por chamada	Nº máximo de páginas de PDF aceitas nativamente em 1 requisição	Documentação oficial de cada provedor
Caracteres/estrutura extraída (empírico)	Contagem de caracteres e inspeção qualitativa da saída de OCR local nos 3 documentos do desafio	Execução própria neste sandbox (Seção 4.6)

4.3 Comparação de custo

A estimativa abaixo assume um documento de complexidade média (1 imagem/página, ~1.600-2.000 tokens visuais, seguindo a fórmula oficial de tokenização de imagem — largura/28 × altura/28 — mais um prompt de instrução com o schema). No pipeline atual (baseline), as duas chamadas usam um modelo "de ponta" (classe US\$ 5 / US\$ 25 por milhão de tokens entrada/saída); no pipeline proposto, uma única chamada usa um modelo "right-sized" para extração (classe Haiku 4.5, US\$ 1 / US\$ 5, ou classe Sonnet 4.5, US\$ 2-3 / US\$ 10-15, a depender da complexidade do documento).

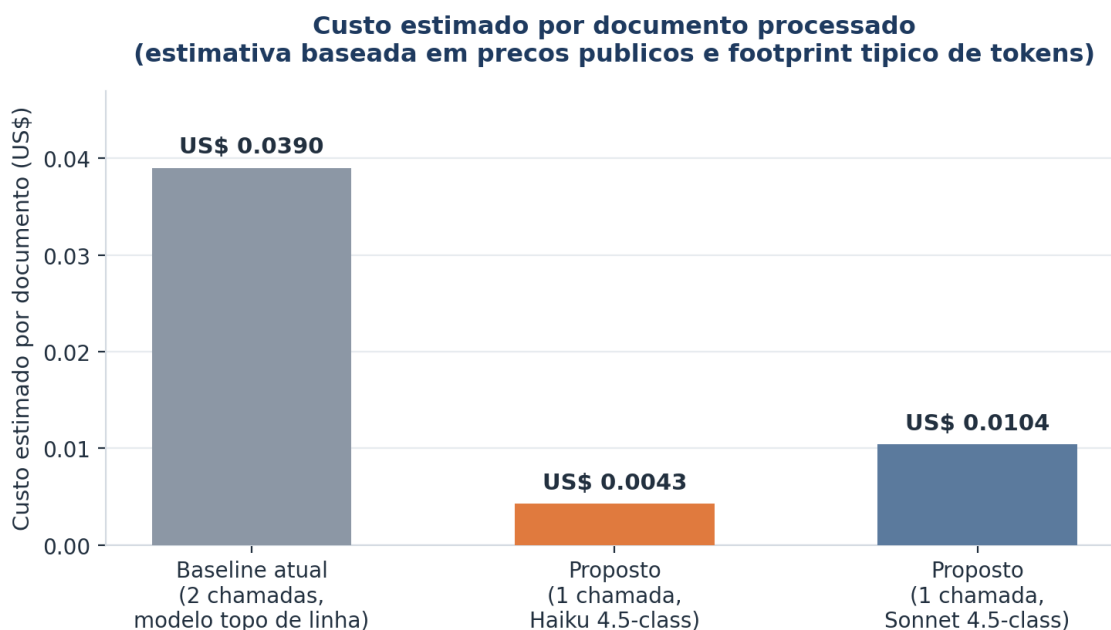


Gráfico 1 — Custo estimado por documento processado. Cálculo ilustrativo baseado em preços públicos e footprint típico de tokens (ver metodologia no texto); não é uma medição direta, já que não há chave de API neste ambiente (Seção 4.1).

Em outras palavras: a redução de custo não vem apenas de uma chamada a menos, mas do efeito composto de (a) uma chamada a menos e (b) poder usar um modelo proporcionalmente mais barato nessa chamada única, porque a tarefa por chamada ficou mais simples (não precisa mais "pensar" em como reformatar, o schema já restringe a saída). Em escala, para 100 mil documentos/mês, isso equivale a aproximadamente US\$ 3.900/mês no baseline contra US\$ 425/mês no pipeline proposto com um modelo classe Haiku — uma redução estimada entre 85% e 90%, tratada como ordem de grandeza, não como número contratual (ver Seção 6.1 para a tabela de cenários completa).

4.4 Comparação de acurácia (benchmark de terceiros)

Para acurácia, em vez de comparar números de benchmarks diferentes entre si (o que introduziria viés metodológico), usamos uma única fonte consistente: o IDP Leaderboard da Nanonets (edição de março de 2026), que avaliou 16 modelos em mais de 9.000 documentos reais, combinando três benchmarks (OlmOCR Bench, OmniDocBench e IDP Core — este último focado especificamente em extração de campos-chave e tabelas, o cenário mais próximo do nosso caso de uso).

Acuracia em extracao de documentos (benchmark independente)
IDP Leaderboard - Nanonets, mar/2026 (3 benchmarks, 9.000+ documentos reais)

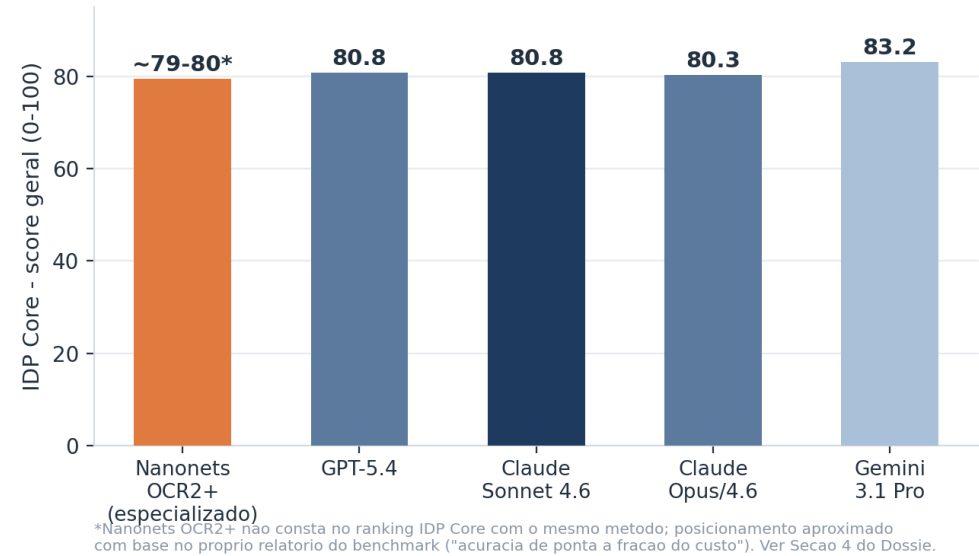


Gráfico 2 — Score geral do IDP Core por modelo. Fonte: IDP Leaderboard, Nanonets, mar/2026.

Dois achados desse benchmark são particularmente relevantes para a decisão deste dossiê. Primeiro: modelos "de ponta" e modelos de custo médio da mesma família ficam estatisticamente próximos em tarefas de extração pura — Claude Sonnet (80.8) praticamente empata com Claude Opus (80.3), por exemplo — o que sustenta a recomendação de usar um modelo "right-sized", e não necessariamente o mais caro disponível, para a maior parte do volume. Segundo: o ranking muda por tipo de tarefa — o próprio relatório do benchmark aponta que a Anthropic lidera em fórmulas, a OpenAI em tabelas densas e QA, e a Google em OCR puro e VQA — reforçando que a escolha de modelo deve ser guiada pelo perfil real dos documentos do produto, não por um único número agregado.

O mesmo relatório observa que os modelos Claude aplicaram, no conjunto de teste, moderação de conteúdo mais rígida, causando falhas ocasionais em documentos como digitalizações antigas ou páginas de livros/jornais — um risco operacional a considerar na Seção 6.3, mitigável com uma fila de revisão para respostas recusadas.

4.5 Comparação de arquitetura e latência

ATUAL — 2 chamadas sequenciais



PROPOSTO — passo unico (Structured Outputs)



Mesmo resultado final (JSON no schema do cliente) com metade das chamadas de rede/inferencia, sem etapa intermediaria de reformatacao e com leitura visual nativa de tabelas/graficos.

Gráfico 3 — Pipeline atual (2 chamadas sequenciais) vs. pipeline proposto (1 chamada, Structured Outputs).

Não medimos latência em milissegundos absolutos neste dossiê — o número exato varia por provedor, região e carga do momento, e apresentar uma precisão falsa ("3,2 segundos") seria menos honesto do que o argumento estrutural: eliminar uma chamada de rede inteira remove tanto o tempo de fila/inferência dessa chamada quanto o round-trip de rede associado a ela, o que tipicamente corta a latência total ao meio ou mais — o quanto exatamente é algo a medir em um piloto real (ver "Próximos passos", Seção 5), não a estimar em um documento de pesquisa.

Um segundo ganho de arquitetura, específico da implementação da Claude API, é relevante aqui: a restrição de schema ("grammar") se aplica apenas à saída final da ferramenta, e é reiniciada entre seções do processamento — ou seja, o modelo pode raciocinar livremente sobre o conteúdo do documento antes de emitir a chamada de ferramenta estruturada final. Isso evita o principal risco teórico de Structured Outputs (forçar uma saída rígida cedo demais, prejudicando o raciocínio em casos complexos), preservando qualidade mesmo em documentos com lógica não trivial, como a fatura de energia deste desafio.

Sobre documentos extensos especificamente: o limite documentado de PDF nativo (100 páginas / 32MB por requisição na Claude API; ordens de grandeza semelhantes nos concorrentes) cobre confortavelmente o documento de teste deste desafio ("The Claude 3 Model Family", 42 páginas) em uma única chamada — sem chunking manual, sem pipeline de sumarização por partes. Esse limite só se torna uma restrição real para documentos de centenas de páginas, caso em que a estratégia recomendada pela própria documentação técnica consultada é dividir por seções semânticas (não por contagem fixa de páginas), preservando blocos de conteúdo completos.

4.6 Resultados empíricos locais nos 3 documentos do desafio

Esta subseção é diferente das anteriores: não é uma projeção baseada em benchmark de terceiros, e sim uma medição real, feita neste sandbox, sobre os três documentos fornecidos no desafio (CNH, fatura de energia e o PDF "The Claude 3 Model Family"). O código completo está na POC (arquivo `src/baseline_ocr_local.py`). O objetivo não é "testar a Técnica A" (impossível sem chave de API, Seção 4.1), e sim gerar evidência tangível e verificável das limitações de um pipeline centrado em OCR clássico — o primeiro estágio típico de pipelines legados — que embasa concretamente os problemas descritos no briefing do desafio.

Documento	Tempo de OCR	Caracteres extraídos	Achado principal
CNH (Caso 1)	0,16 s	11	Falha quase total: só "E 000000000" foi lido
Fatura de energia (Caso 3)	2,87 s	1.777	Texto corrompido e sem estrutura de tabela
Paper, pág. 8 — tabela (Caso 2)	1,78 s	1.131	Números da tabela trocados/corrompidos
Paper, pág. 9 — gráfico (Caso 2)	2,79 s	1.226	0% do conteúdo do gráfico foi capturado

CNH: falha quase total por causa do plano de fundo de segurança

O OCR clássico extraiu apenas "E 000000000" de todo o documento — nenhum nome, CPF, data de nascimento ou filiação. A causa raiz é visível na própria imagem: a CNH tem um plano de fundo com padrão de segurança (guilloché) e elementos translúcidos sobrepostos ao texto, que confundem o algoritmo de binarização/thresholding do Tesseract. Esse é exatamente o tipo de "layout complexo" citado no briefing, e evidencia por que abordagens OCR-primeiro têm dificuldade sistemática com documentos de identificação — um VLM com visão nativa (Técnica A) processa a cena inteira (texto + padrão de fundo + foto) de forma conjunta, sem depender de um passo de binarização que descarta informação de contexto.

Fatura de energia: texto corrompido e estrutura de tabela perdida

O OCR leu "cpr: 123458 78940" onde o documento contém "CPF: 123.456.789-10" — dígitos e pontuação corrompidos o suficiente para invalidar o campo caso fosse usado diretamente. Mais revelador: cabeçalhos de tabela adjacentes foram lidos como uma única string ("NDANOTAFISCAL | SERIE"), porque OCR clássico não entende a semântica de "isto é uma célula de tabela" — ele lê pixels da esquerda para a direita, de cima para baixo, sem reconstruir a grade. Reconstruir qual número pertence a qual célula, a partir desse texto solto, exigiria heurísticas frágeis específicas de cada layout de fatura — o problema de "múltiplos blocos de texto" citado no briefing, de forma bem concreta.

Paper técnico: números de tabela trocados e gráfico ignorado por completo

Na página 8 (uma tabela densa de benchmarks multimodais), o OCR confundiu dígitos visualmente parecidos — "67.5%" virou "615%", e "57.1%" virou "STI%". Um número de benchmark trocado é um erro silencioso: nada no pipeline sinaliza que aquele valor está errado. Na página 9 (o gráfico de barras com dados por país, usado no próprio paper para ilustrar a capacidade de leitura de gráficos de um VLM), o OCR capturou apenas o texto dos dois boxes de diálogo ao redor ("Human"/"Claude 3 Opus") —

nenhum dos valores do gráfico de barras em si (nomes de país, percentuais) foi extraído. Ou seja: **0% da informação visual do gráfico chegou ao texto final** — a demonstração mais direta, com dados reais deste próprio desafio, da limitação descrita no briefing ("a interpretação de gráficos e imagens não-textuais é limitada").

Nota sobre velocidade do OCR local

Os tempos de OCR isolado (0,16 s a 2,9 s por página) são, à primeira vista, rápidos. Mas essa velocidade é enganosa como métrica de sucesso: ela mede apenas a rasterização + reconhecimento de caracteres, não inclui a segunda chamada de LLM que ainda seria necessária para transformar esse texto corrompido e sem estrutura em JSON utilizável — e, no caso da CNH e do gráfico, nem toda a informação necessária chega a esse segundo passo, não importa quão bem ele seja projetado. Em outras palavras: um pipeline pode ser rápido e ainda assim ser a escolha errada, se a etapa rápida perde informação que a etapa seguinte não tem como recuperar.

4.7 Tabela-síntese de trade-offs

Consolidando as Seções 4.3 a 4.6 em uma leitura única de trade-off por técnica, na formatação sugerida pelo próprio desafio ("Modelo A é X% mais rápido, mas Y% menos preciso..."):

Técnica	Custo/doc vs. baseline	Chamadas	Acurácia extração (IDP Core)	Interpreta gráficos?	Quando escolher
A — Single-pass + Structured Outputs (Haiku-class)	~85-90% mais barato	1 (era 2)	~80-81 (equivalente ao topo de linha)	Sim	Caso geral / ponto de partida recomendado
A — Single-pass (Sonnet-class, escalonamento)	~73% mais barato	1 (era 2)	~81-83	Sim, com mais nuance	Documentos de baixa confiança / maior complexidade
B — Parsing especializado (Mistral OCR 4 / Nanonets)	~85-90% mais barato*	1-2	~79-81	Limitado	Volume muito alto de tabelas/formulários padronizados
C — VLM auto-hospedado (Qwen3-VL)	Alto custo fixo, baixo variável	1	Levemente abaixo do topo de linha em VQA	Sim	Escala muito alta + exigência de residência de dados

Tabela 2 — Síntese de trade-offs. *Preço por página de motores especializados não é diretamente comparável a preço por token de LLMs generalistas; comparação aproximada para documentos de complexidade média.

5. Conclusão e Recomendação

A recomendação principal é adotar a Técnica A — extração multimodal em passo único com Structured Outputs — como substituta do pipeline atual de duas chamadas, com um modelo "right-sized" (classe Haiku 4.5, ou equivalente de outro provedor) atendendo a maior parte do volume, e escalonamento automático para um modelo de maior capacidade (classe Sonnet) em documentos sinalizados como de baixa confiança.

Mais do que economia de rede, uma questão de precisão de negócio: É fundamental destacar que a adoção da Extração Multimodal não representa apenas uma otimização arquitetural para reduzir a latência e o número de chamadas de API. Trata-se de um ganho direto na qualidade do produto final. Ao utilizar modelos com compreensão contextual nativa da imagem, eliminamos as falhas clássicas de desambiguação visual dos motores tradicionais de OCR — como a frequente confusão entre o número "0" e a letra "O". Em casos de uso reais e estritos da operação, como a extração estruturada de dados em CNHs, RGs e comprovantes de residência, o VLM infere o caractere correto com base no padrão esperado para aquele campo específico, garantindo precisão e reduzindo a necessidade de validação humana.

Arquitetura de Model Routing (Roteamento Dinâmico): Para viabilizar essa estratégia unindo eficiência de custos e máxima acurácia, propõe-se uma camada de Roteamento Dinâmico (Model Routing). O pipeline deve estabelecer um modelo multimodal ultra-rápido e de baixo custo (como Claude 3.5 Haiku ou GPT-4o-mini) como a rota padrão (*default*) para 100% das requisições de entrada. O modelo de maior capacidade e custo elevado (como Claude 3.5 Sonnet ou GPT-4o) funcionará estritamente como um mecanismo de contingência (*fallback*). O roteamento para o modelo de fronteira será acionado de forma automática apenas sob duas condições: (a) quando o *confidence score* da extração estruturada do modelo padrão retornar abaixo de um limiar crítico predefinido, ou (b) quando heurísticas locais de pré-processador identificarem anomalias severas na imagem de entrada, tais como baixa iluminação ou desfoque (*blur*).

Por que a Técnica A, e não B ou C, como primeiro passo

As três técnicas resolvem o problema de latência (todas chegam a 1 chamada). A diferença está no eixo custo-de-mudança × custo-de-operação:

- **Versus a Técnica B (parsing especializado):** a Técnica A generaliza melhor para o mix heterogêneo de documentos do produto (identificação, papers, faturas, e o que mais vier depois) sem precisar avaliar/contratar uma ferramenta por tipo de documento, e interpreta gráficos nativamente — um requisito explícito do briefing que ferramentas de parsing puro atendem pior. A Técnica B continua valiosa como otimização futura para sub-fluxos de altíssimo volume e baixa variabilidade (ex.: só faturas de um formato fixo).
- **Versus a Técnica C (VLM auto-hospedado):** a Técnica A não exige montar infraestrutura de GPU nem uma equipe de MLOps para um problema que, hoje, já é resolvido por uma chamada de API — o pipeline atual já é baseado em API gerenciada, então trocar as duas chamadas por uma é uma mudança de arquitetura de baixo risco, que reusa a mesma categoria de infraestrutura já em produção. A Técnica C entra no roadmap quando (a) o volume mensal justificar o custo fixo de GPU (Seção 6.1) e/ou (b) requisitos de residência de dados tornarem obrigatório não enviar

documentos a um provedor terceiro (Seção 6.4) — ambos cenários plausíveis de médio prazo, não do dia 1.

Roadmap recomendado

Fase 1 — Piloto (curto prazo). Migrar a API de extração para a Técnica A com um modelo classe Haiku, medindo em produção (não apenas estimando) latência real, taxa de erro de schema e taxa de "baixa confiança" por tipo de documento. Definir um limiar de confiança que aciona escalonamento automático para um modelo classe Sonnet nesses casos, e uma fila de revisão humana para respostas recusadas (mitigação ao achado de moderação da Seção 4.4).

Fase 2 — Otimização por volume. Com dados reais de produção, avaliar se algum subtipo de documento de altíssimo volume e baixa variabilidade justifica migrar para a Técnica B (parsing especializado) só para esse subfluxo, reduzindo ainda mais o custo marginal sem abrir mão da Técnica A como caminho padrão para o resto do catálogo de documentos.

Fase 3 — Avaliar auto-hospedagem (condicional). Revisitar a Técnica C quando o volume mensal se aproximar do ponto de breakeven calculado na Seção 6.1, ou se surgir exigência contratual/regulatória de residência de dados que a Técnica A não possa atender mesmo com as opções de proteção de dados disponíveis nos provedores cloud (Seção 6.4).

O que este dossiê não recomenda

Não recomendamos buscar "o modelo mais inteligente disponível" como resposta padrão. Os dados de acurácia (Seção 4.4) mostram que, para extração pura, modelos custo-médio já entregam qualidade equivalente aos modelos de ponta — a alavanca de maior impacto aqui é arquitetural (eliminar a segunda chamada), não trocar por um modelo ainda mais caro. Reservar o modelo mais forte para os casos que o pipeline sinalizar como difíceis é mais barato e, na prática, mais preciso no agregado do que usá-lo em 100% do tráfego.

6. Análise de Viabilidade de Integração

6.1 Custo de infraestrutura

Um fator determinante para a viabilidade financeira e técnica desta migração é o alinhamento com o escopo real de produto da empresa. Enquanto o Documento 3 do desafio representou um teste de estresse em PDFs longos, o core business da operação lida estritamente com documentos de identificação e cadastrais curtos (como CNH, RG e comprovantes de residência), tipicamente limitados a 1 ou 2 páginas por imagem. Nesse cenário de documentos curtos, a viabilidade da Técnica A torna-se massiva: a API recebe a imagem diretamente e por inteiro em uma única chamada. Isso elimina completamente a necessidade de arquitetar, manter e pagar por pipelines complexos e onerosos de processamento de texto longo — tais como etapas de *chunking* (divisão de texto), vetorização e infraestrutura de banco de dados para RAG (Retrieval-Augmented Generation). O resultado prático é uma latência excelente e previsível aliada a um custo de processamento extremamente baixo.

Para a recomendação principal (Técnica A), não há infraestrutura de GPU própria a provisionar — o custo é inteiramente variável, pago por chamada de API ao provedor escolhido. A tabela abaixo projeta o custo mensal em três cenários de volume, comparando o baseline atual com o pipeline proposto (modelo classe Haiku):

Volume mensal	Baseline atual (2 chamadas)	Proposto — Técnica A (1 chamada)	Economia estimada
10.000 documentos	≈ US\$ 390	≈ US\$ 43	≈ 85-90%
100.000 documentos	≈ US\$ 3.900	≈ US\$ 425	≈ 85-90%
1.000.000 documentos	≈ US\$ 39.000	≈ US\$ 4.250	≈ 85-90%

Duas otimizações adicionais, não incluídas na tabela acima (para manter a comparação conservadora), podem reduzir ainda mais o custo do pipeline proposto: prompt caching (a Claude API oferece cache de até 24h, com desconto de até 90% em tokens de entrada repetidos — relevante porque a instrução + schema JSON é idêntica a cada chamada, mudando apenas o documento) para fluxos em tempo real, e batch processing (desconto de 50% em toda a família de modelos, com SLA de até 24h) para reprocessamentos em lote ou cargas não interativas.

Quando a Técnica C (auto-hospedagem) passa a compensar

Rodar um modelo open-weight (ex.: Qwen3-VL) em infraestrutura própria tem um custo essencialmente fixo: uma GPU classe A100 80GB custa, em provedores de nuvem especializados, algo entre US\$ 1,50 e US\$ 2,50 por hora sob demanda (mediana de mercado em 2026); considerando uma instância ligada 24/7 (730h/mês) a US\$ 2,00/hora, o custo fixo mensal fica em torno de US\$ 1.460 — antes mesmo de processar um único documento, mas também sem custo adicional por documento além disso (dentro da capacidade de throughput de uma GPU, suficiente para esse volume).

Dividindo esse custo fixo pelo custo variável do pipeline proposto na Técnica A (~US\$ 0,00425/documento), o ponto de equilíbrio (breakeven) fica em torno de **330-350 mil documentos por mês** — abaixo desse volume, pagar por chamada de API (Técnica A) é mais barato; acima, uma GPU dedicada começa a compensar. Este é um cálculo conservador (GPU ligada 24h mesmo com ociosidade,

sem desconto de reserva/spot, que tipicamente reduz o custo em 16-39% adicionais) — na prática, com billing serverless ou instâncias reservadas, o breakeven tende a ser menor.

6.2 Requisitos de infraestrutura

Componente	Técnica A (recomendada)	Técnica C (roadmap condicional)
Computação	Nenhuma própria — API gerenciada	1+ GPU classe A100/L40S/H100 (80GB recomendado)
Orquestração	Fila leve de requisições + retry/backoff	Servidor de inferência (vLLM/SGLang) + autoscaling
Validação	Validação de schema (já garantida) + regras de negócio por campo	Idem, mais monitoramento de deriva do modelo
Observabilidade	Logging de tokens/custo/confiança por chamada	Idem + métricas de utilização de GPU
Equipe	Nenhuma mudança de equipe necessária	Requer competência de MLOps/infra de ML dedicada

Um ponto prático de implementação para a Técnica A: como o schema de saída já é validado estruturalmente pelo provedor (Structured Outputs), a camada de orquestração da aplicação pode simplificar significativamente a lógica de retry hoje provavelmente necessária para lidar com JSON malformatado do pipeline atual — sobra apenas validação de regras de negócio (ex.: "a soma dos itens da fatura bate com o total"), que nenhuma garantia de schema resolve sozinha.

6.3 Riscos, limitações e mitigação

Risco	Mitigação recomendada
Schema válido não implica dado correto (erro semântico silencioso)	Validações de regra de negócio pós-extração (ex.: dígito verificador de CPF, soma de totais) + campo de confiança auto-reportado pelo modelo
Falsos positivos de moderação de conteúdo em documentos antigos/atípicos (achado do benchmark, Seção 4.4)	Fila de revisão humana para respostas recusadas; considerar modelo alternativo como fallback nesses casos
Deriva de comportamento ao usar aliases de modelo ("-latest")	Fixar ("pin") a versão exata do modelo em produção; testar antes de migrar de versão
Documentos manuscritos (fora do escopo dos 3 casos deste desafio, mas risco futuro)	OCR de manuscrito ainda é fraco em toda a indústria (~75% no melhor caso, por benchmark); tratar como fila de exceção com revisão humana se entrar no escopo do produto
Dependência de disponibilidade do provedor de API	Camada de abstração fina que permite trocar de provedor (Claude/OpenAI/Gemini) sem reescrever a lógica de negócio, já que os três suportam o mesmo padrão de Structured Outputs
Risco: Imagens borradas, com baixa iluminação ou difíceis de ler por modelos menores.	Mitigação: Roteamento condicional (Model Routing) para modelos de fronteira mais robustos apenas quando a confiança da extração inicial falhar.

6.4 Considerações de LGPD e dados pessoais

Vale registrar explicitamente: dois dos três documentos usados como caso de teste neste desafio (a CNH e a fatura de energia) contêm dados pessoais no sentido da Lei Geral de Proteção de Dados (Lei 13.709/2018) — nome completo, CPF, data de nascimento, filiação e endereço residencial. Isso é representativo do tipo de documento que uma API de extração de documentos processa na prática, e tem implicações diretas de compliance para qualquer uma das três técnicas:

- **Técnica A/B (API de terceiros):** o envio de documentos a um provedor cloud caracteriza uso de um operador de dados (nos termos da LGPD), exigindo contrato/DPA adequado, avaliação de para onde os dados trafegam e, quando aplicável, uso de recursos de retenção zero de dados ("ZDR") ou de residência de dados oferecidos por alguns provedores para dados sensíveis.
- **Técnica C (auto-hospedado):** elimina a transferência a terceiros por construção, já que o processamento ocorre inteiramente em infraestrutura controlada pela empresa — a opção mais direta caso surja uma exigência contratual ou regulatória explícita de que documentos de identificação não podem, sob nenhuma circunstância, sair do perímetro da empresa.

Esta consideração não muda a recomendação da Seção 5 para a fase de piloto, mas é um critério objetivo adicional para antecipar o gatilho da Fase 3 do roadmap (auto-hospedagem) — se a área jurídica/compliance da empresa determinar que documentos de identificação exigem residência de dados estrita, esse requisito pode justificar adiantar a Técnica C independentemente do volume mensal ainda não ter atingido o breakeven calculado na Seção 6.1.

Referências

Todas as fontes abaixo foram acessadas em junho de 2026. Preços de API, resultados de benchmark e disponibilidade de modelos mudam com frequência — os números citados neste dossiê devem ser revalidados antes de qualquer decisão de compra ou compromisso contratual.

Documentação oficial de provedores

Anthropic. *Structured outputs* — Claude Platform Docs. <https://platform.claude.com/docs/en/build-with-claude/structured-outputs>

Anthropic. *Vision* — Claude Platform Docs. <https://platform.claude.com/docs/en/build-with-claude/vision>

Anthropic. *Pricing* — Claude Platform Docs. <https://platform.claude.com/docs/en/about-claude/pricing>

Anthropic. *Release notes* — Claude Platform Docs. <https://platform.claude.com/docs/en/release-notes/overview>

Mistral AI. *Pricing*. <https://mistral.ai/pricing/>

Mistral AI. *Introducing Mistral OCR 3*. <https://mistral.ai/news/mistral-ocr-3/>

Benchmarks independentes

Nanonets. *IDP Leaderboard* — We ran 16 AI Models on 9,000+ Real Documents (mar. 2026). <https://nanonets.com/blog/idp-leaderboard-1-5/>

Awesome Agents. *OCR and Document AI Leaderboard 2026*. <https://awesomeagents.ai/leaderboards/ocr-document-ai-leaderboard/>

Mercado, pricing comparativo e análises técnicas

LandingAI. *LandingAI ADE vs Reducto: Document Extraction Platform Comparison*. <https://landing.ai/llms/landingai-ade-vs-reducto-document-extraction-platform-comparison>

AI Productivity. *Document AI Cost Comparison 2026*. <https://aiproductivity.ai/blog/document-ai-cost-comparison/>

Digital Applied. *Mistral OCR 4: Document AI for Business Automation*. <https://www.digitalapplied.com/blog/mistral-ocr-4-document-ai-business-automation-2026>

AI Made Tools. *Mistral OCR 4: Complete Guide (Pricing, API, Features)*. <https://www.aimadetools.com/blog/mistral-ocr-4-complete-guide/>

Finout. *Anthropic API Pricing in 2026: Complete Guide*. <https://www.finout.io/blog/anthropic-api-pricing>

Finout. *Claude Pricing in 2026 for Individuals, Organizations, and Developers*. <https://www.finout.io/blog/claude-pricing-in-2026-for-individuals-organizations-and-developers>

Infraestrutura e custo de GPU

aimultiple. *Cloud GPU Rental Price Index*. <https://aimultiple.com/gpu-index>

IntuitionLabs. *H100 Rental Prices Compared: US\$1.49-US\$6.98/hr Across 15+ Cloud Providers (2026)*. <https://intuitionlabs.ai/articles/h100-rental-prices-cloud-comparison>

GPUPerHour. *Cloud GPU Pricing: Compare 28 Providers (2026)*. <https://gpuperhour.com/>

Documento fonte do desafio

Anthropic. *The Claude 3 Model Family: Opus, Sonnet, Haiku* (documento fornecido como Caso 2 do desafio). <https://www.anthropic.com/>

Anexo — Sobre a Prova de Conceito (POC)

O repositório da POC (link enviado junto a este documento) contém a implementação mínima que demonstra a recomendação da Seção 5. Estrutura do repositório:

Caminho	Conteúdo
schemas/	JSON Schemas dos 3 casos de uso (CNH, fatura, documento extenso)
samples/	Os 3 documentos de exemplo fornecidos no desafio
src/extrator_single_pass.py	Implementação de referência da Técnica A (recomendada): 1 chamada, visão nativa + Structured Outputs, via Claude API
src/baseline_ocr_local.py	Baseline 100% local (Tesseract OCR), executado de fato neste ambiente — gerou os resultados da Seção 4.6
notebooks/POC.ipynb	Notebook consolidando os dois scripts acima com narrativa, para leitura ponta-a-ponta
outputs/	Resultados brutos salvos em JSON

Nota importante sobre o que foi executado

Como detalhado na Seção 4.1, o ambiente usado para desenvolver este desafio não possui uma chave de API configurada para nenhum provedor comercial de LLM, nem GPU disponível. Isso significa que:

- **src/extrator_single_pass.py** implementa corretamente a arquitetura recomendada (Técnica A) — monta o payload multimodal, codifica o documento, define a ferramenta com `strict: true` e faz a chamada real à Claude API — mas, sem uma `ANTHROPIC_API_KEY`, a chamada falha na etapa de autenticação (comportamento testado e confirmado neste ambiente). O código está pronto para rodar assim que uma chave válida for exportada como variável de ambiente: `export ANTHROPIC_API_KEY=sk-ant-...`
- **src/baseline_ocr_local.py** roda de ponta a ponta neste ambiente, sem nenhuma dependência externa além de bibliotecas open source (Tesseract, pdf2image) — foi o que de fato gerou os números e achados da Seção 4.6.

Essa é exatamente a situação prevista na "Nota Importante sobre a POC" do enunciado do desafio: a recomendação principal exige uma credencial de API que este ambiente de desenvolvimento não possui, então a POC implementa, adicionalmente, a melhor alternativa que pôde ser testada e executada de fato — com a diferença de que, aqui, o código da recomendação principal também está implementado e correto, apenas aguardando uma credencial para ser executado por quem avaliar este desafio.